Лекция 6. Коллективные коммуникации: работа с векторами

13 октября 2026 г.

Для распределения массива между процессами и последующего сбора результатов для вывода используются следующие функции коллективной коммуникации:

```
int MPI_Scatter(
    void*           send_buf_p,     /*in*/
    int             send_count,     /*in*/
    MPI_Datatype    datatype,       /*in*/
    void*           recv_buf_p,     /*out*/
    int             recv_count,     /*in*/
    MPI_Datatype    recv_type,      /*in*/
    int             src_proc,       /*in*/
    MPI_Comm        comm            /*in*/
);
```

Функция `Scatter` разделяет переданный ей массив на процессе `src_proc` на некоторое количество частей по `send_count = recv_count` элементов в каждом и отправляет их всем процессам в коммуникаторе.

```
int MPI_Gather(
    void*           send_buf_p,     /*in*/
    int             send_count,     /*in*/
    MPI_Datatype    datatype,       /*in*/
    void*           recv_buf_p,     /*out*/
    int             recv_count,     /*in*/
    MPI_Datatype    recv_type,      /*in*/
    int             dest_proc,      /*in*/
```

```
    MPI_Comm        comm            /*in*/
);
```

Функция `MPI_Gather` берет массивы `send_buf_p` с каждого процесса в коммуникаторе и собирает их в единый массив `recv_buf_p` на процессе `dest_proc`.

Также есть функция `MPI_Allgather`, сигнатура которой отличается от `MPI_Gather` лишь отсутствием `dest_proc`. Данная функция является последовательностью вызовов `MPI_Gather` и `MPI_Bcast`.

С уществуют варианты вышеприведенных функций, позволяющие разделять массив на неравные части. Они позволяют решать задачи без предположения, что размер вектора или матрицы делится нацело на количество процессов.

```
int MPI_Scatterv(
        const void *sendbuf,      // in
        const int sendcounts[],   // in
        const int displs[],       // in
        MPI_Datatype sendtype,    // in
        void *recvbuf,            // out
        int recvcount,            // in
        MPI_Datatype recvtype,    // in
        int root,                 // in
        MPI_Comm comm             // in
);
```

Принципиальными являются второй и третий аргументы. Оба массива имеют длину, равную количеству процессов. Элемент с индексом i массива `sendcounts` содержит длину подмассива `sendbuf`, которая достается процессу i. Элемент с индексом i массива `displs` содержит начальный индекс `sendbuf`, с которого отсчитывается `sendcounts[i]` элементов.

```
int MPI_Gatherv(
        const void *sendbuf,      // in
        int sendcount,            // in
        MPI_Datatype sendtype,    // in
        void *recvbuf,            // out
        const int recvcounts[],   // in
        const int displs[],       // in
```

```
        MPI_Datatype recvtype,   // in
        int root,                // in
        MPI_Comm comm            // in
);
```

Пример 1. Написать программу, осуществляющую распределенное умножение матрицы на вектор-столбец. Матрицу разбивать на строки. Каждому процессу достается какое-то количество строк матрицы и весь столбец. Предполагается, что количество строк делится нацело на количество процессов.

```
1  #include <mpi.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  void InputDims(int my_rank, int *r, int *c);
6  void InputVec(int my_rank, int n, int* vec);
7  void InputMatrix(int my_rank, int r, int local_r, int c, int *mat);
8  void MatVecMult(int local_r, int *mat, int *local_res,
9    int *global_res, int c, int *vec);
10 void PrintVec(int my_rank, int *v, int n);
11
12
13 int main(){
14
15   int my_rank;
16   int comm_sz;
17
18   MPI_Init(NULL, NULL);
19
20   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
21   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
22
23   int r,c;
24   InputDims(my_rank, &r, &c);
25
26   int *v = calloc(c, sizeof(int));
27   InputVec(my_rank, c, v);
28
```

```
29    int *m = calloc(r / comm_sz * c, sizeof(int));
30    InputMatrix(my_rank, r, r / comm_sz, c, m);
31    int *local_res = calloc(r / comm_sz, sizeof(int));
32    int *global_res = NULL;
33    if(my_rank == 0)
34      global_res = calloc(r, sizeof(int));
35    MatVecMult(r / comm_sz, m, local_res, global_res, c, v);
36
37    PrintVec(my_rank, global_res, r);
38
39    MPI_Finalize();
40
41  }
42
43  void InputDims(int my_rank, int *r, int *c){
44
45    if(my_rank == 0){
46      scanf("%d", r);
47      scanf("%d", c);
48    }
49    MPI_Bcast(r, 1, MPI_INT, 0, MPI_COMM_WORLD);
50    MPI_Bcast(c, 1, MPI_INT, 0, MPI_COMM_WORLD);
51
52  }
53
54  void InputVec(int my_rank, int n, int* vec){
55
56    if(my_rank == 0){
57      printf("v = ");
58      for(int i = 0; i < n; i++){
59        vec[i] = rand() % 5;
60        printf("%d ", vec[i]);
61      }
62      printf("\n");
63    }
64    MPI_Bcast(vec, n, MPI_INT, 0, MPI_COMM_WORLD);
65
66  }
```

```
67
68 void InputMatrix(int my_rank, int r, int local_r, int c, int *mat){
69
70   if(my_rank == 0){
71     printf("M = \n");
72     int *temp = calloc(r*c, sizeof(int));
73     for(int i = 0; i < r*c; ++i){
74       if(i % c == 0 && i != 0)
75       printf("\n");
76       temp[i] = rand() % 5;
77       printf("%d ", temp[i]);
78     }
79     printf("\n");
80     MPI_Scatter(temp, local_r * c, MPI_INT, mat, local_r * c,
81       MPI_INT, 0, MPI_COMM_WORLD);
82     free(temp);
83   }
84   else
85     MPI_Scatter(NULL, local_r * c, MPI_INT, mat,
86       local_r * c, MPI_INT, 0, MPI_COMM_WORLD);
87
88 }
89
90 void MatVecMult(int local_r, int *mat, int *local_res,
91   int *global_res, int c, int *vec){
92
93   for(int i = 0; i < local_r; ++i){
94     for(int j = 0; j < c; ++j){
95       local_res[i] += mat[i * c + j] * vec[j];
96     }
97   }
98   MPI_Gather(local_res, local_r, MPI_INT,
99     global_res, local_r, MPI_INT, 0, MPI_COMM_WORLD);
100
101 }
102
103 void PrintVec(int my_rank, int *v, int n){
104
```

```
105     if(my_rank == 0){
106       printf("Res = ");
107       for(int i = 0; i < n; ++i)
108         printf("%d ", v[i]);
109       printf("\n");
110     }
111
112   }
```

Комментарии к коду.

1. Функция `InputDims` в нулевом процессе осуществляет ввод с клавиатуры количества строк `r` и количества столбцов `c` матрицы. Затем эта информация с помощью функции `MPI_Bcast` рассылается всем процессам.

2. Функция `InputVec` в нулевом процессе заполняет переданный ей массив `vec` и рассылает его всем процессам. Выделение памяти под массив в этой функции не производится.

3. В функции `InputMatrix` на нулевом процессе создается временный массив длиной `rc`. Хотя массив одномерный, он будет использоваться для хранения матрицы. Массив заполняется случайными числами. С помощью функции `MPI_Scatter` массив разбивается на `n` равных частей, каждая из которых отправляется отдельному процессу. Каждая часть содержит local_r $= r/$comm_sz строк матрицы. Часть матрицы сохраняется в переменной `mat` на каждом процессе.

   Обратите внимание, что мы вызываем функцию `MPI_Scatter` в 80 строке на нулевом процессе и для любого другого процесса в строке 85. Это сделано для того, чтобы выделять память под временный массив `temp` только на нулевом процессе. Т.к. коллективные коммуникации нужно вызывать на всех процессах коммуникатора, то возникает необходимость написать отдельный вызов для ненулевых процессов. В качестве первого аргумента передается нулевой указатель, т.к. он релевантен только для корневого (в нашем случае нулевого) процесса.

4. В функции `MatVecMult` происходит умножение части матрицы на вектор-столбец, результатом чего является часть итогового векто-

ра — произведения матрицы на столбец. Части результирующего вектора собираются воедино в строке 98, в которой функция `MPI_Gather` собирает их в единый массив на нулевом процессе.

Пример 2. Решить предыдущую задачу без допущения о том, что количество строк делится без остатка на количество процессов.

```
 1 #include <mpi.h>
 2 #include <stdio.h>
 3 #include <stdlib.h>
 4
 5 void InputDims(int my_rank, int *r, int *c);
 6 void InputVec(int my_rank, int n, int* vec);
 7 void InputMatrix(int my_rank, int r, int c, int *mat,
 8   int *sizes, int *displs);
 9 void MatVecMult(int *mat, int *local_res, int *global_res,
10   int c, int *vec, int *sizes, int *displs, int my_rank);
11 void PrintVec(int my_rank, int *v, int n);
12 void BuildSizes(int comm_sz, int r, int c, int *sizes);
13 void BuildDispls(int comm_sz, int *displs, int *sizes);
14
15 int main(){
16
17   int my_rank;
18   int comm_sz;
19
20   MPI_Init(NULL, NULL);
21
22   MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
23   MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
24
25   int r, c;
26   InputDims(my_rank, &r, &c);
27
28   int *v = calloc(c, sizeof(int));
29   InputVec(my_rank, c, v);
30
31   int *sizes_v = calloc(comm_sz, sizeof(int));
32   int *displs_v = calloc(comm_sz, sizeof(int));
```

```
33    int *sizes_m = calloc(comm_sz, sizeof(int));
34    int *displs_m = calloc(comm_sz, sizeof(int));
35
36    BuildSizes(comm_sz, r, 1, sizes_v);
37    BuildSizes(comm_sz, r, c, sizes_m);
38    BuildDispls(comm_sz, displs_v, sizes_v);
39    BuildDispls(comm_sz, displs_m, sizes_m);
40
41
42    int *m = calloc(r / comm_sz * c, sizeof(int));
43    InputMatrix(my_rank, r, c, m, sizes_m, displs_m);
44    int *local_res = calloc(r / comm_sz, sizeof(int));
45    int *global_res = NULL;
46    if(my_rank == 0)
47      global_res = calloc(r, sizeof(int));
48    MatVecMult(m, local_res, global_res, c, v, sizes_v,
49      displs_v, my_rank);
50
51    PrintVec(my_rank, global_res, r);
52
53    MPI_Finalize();
54
55  }
56
57  void BuildSizes(int comm_sz, int r, int c, int *sizes){
58    for(int i = 0; i < comm_sz; ++i){
59      sizes[i] = r / comm_sz;
60      if(i < r % comm_sz)
61      sizes[i]++;
62      sizes[i] *= c;
63    }
64  }
65
66  void BuildDispls(int comm_sz, int *displs, int *sizes){
67    displs[0] = 0;
68    for(int i = 1; i < comm_sz; ++i){
69      displs[i] = displs[i-1] + sizes[i-1];
70    }
```

```
71 }
72
73 void InputDims(int my_rank, int *r, int *c){
74
75   if(my_rank == 0){
76     scanf("%d", r);
77     scanf("%d", c);
78   }
79   MPI_Bcast(r, 1, MPI_INT, 0, MPI_COMM_WORLD);
80   MPI_Bcast(c, 1, MPI_INT, 0, MPI_COMM_WORLD);
81
82 }
83
84 void InputVec(int my_rank, int n, int* vec){
85
86   if(my_rank == 0){
87     printf("v = ");
88     for(int i = 0; i < n; i++){
89       vec[i] = rand() % 5;
90       printf("%d ", vec[i]);
91     }
92     printf("\n");
93   }
94   MPI_Bcast(vec, n, MPI_INT, 0, MPI_COMM_WORLD);
95
96 }
97
98 void InputMatrix(int my_rank, int r, int c, int *mat,
99   int *sizes, int *displs){
100
101   if(my_rank == 0){
102     printf("M = \n");
103     int *temp = calloc(r*c, sizeof(int));
104     for(int i = 0; i < r*c; ++i){
105       if(i % c == 0 && i != 0)
106       printf("\n");
107       temp[i] = rand() % 5;
108       printf("%d ", temp[i]);
```

```
109       }
110       printf("\n");
111       MPI_Scatterv(temp, sizes, displs, MPI_INT, mat,
112         sizes[my_rank], MPI_INT, 0, MPI_COMM_WORLD);
113       free(temp);
114     } else
115     MPI_Scatterv(NULL, sizes, displs, MPI_INT,
116       mat, sizes[my_rank], MPI_INT, 0, MPI_COMM_WORLD);
117
118 }
119
120 void MatVecMult(int *mat, int *local_res, int *global_res,
121       int c, int *vec, int *sizes, int *displs, int my_rank){
122     for(int i = 0; i < sizes[my_rank]; ++i){
123       for(int j = 0; j < c; ++j){
124         local_res[i] += mat[i * c + j] * vec[j];
125       }
126       if(my_rank == 0)
127       printf("r=%d ", local_res[i]);
128     }
129     MPI_Gatherv(local_res, sizes[my_rank], MPI_INT, global_res,
130       sizes, displs, MPI_INT, 0, MPI_COMM_WORLD);
131
132 }
133
134 void PrintVec(int my_rank, int *v, int n){
135
136     if(my_rank == 0){
137       printf("Res = ");
138       for(int i = 0; i < n; ++i)
139       printf("%d ", v[i]);
140       printf("\n");
141     }
142
143 }
```

Комментарии к коду. Основным отличием данного листинга от предыдущего является использование функций `MPI_Scatterv` и `MPI_Ga-`

`therv`, вместо `MPI_Scatter` и `MPI_Gather`. Для вызова этих функций требуется определить массивы размеров и смещений. Эти массивы заполняются в функциях `BuildSizes` и `BuildDispls` соответственно. Обратите внимание на то, что нам потребуется два массива размеров (и, соответственно, смещений, т.к. массив смещений определяется с использованием массива размеров) отдельно для ввода и расчленения матрицы в строке 43 и для склеивания частичных результатов умножения в единый вектор в строках 48 – 49. С этой целью мы определяем функцию `BuildSizes` для матрицы, а используем ее как для матрицы (размером `r x c`), так и для вектора (размер `n x 1`).